

Computação Gráfica e Processamento de Imagens

Processamento digital de imagens

Ma. Vanessa Matias Leite

1

- Unidade de Ensino: 04
- Competência da Unidade: Compreender os fundamentos das imagens, sabendo analisar um problema de computação gráfica e classificá-lo
- Resumo: Saber reconhecer e aplicar a segmentação de imagens usando watershed e filtros no domínio espacial e da frequência.
- Palavras-chave: filtros, imagem, Sobel, Laplaciano, watershed
- Título da Teleaula: Processamento Digital de Imagens.
- Teleaula nº: 04

2

Filtros de Imagens Digitais

3

Filtro de imagens digitais

Processar uma imagem significa alterar os valores de seus pixels para se criar um novo, visando a uma alteração do conteúdo dessa imagem;

- ✓ Efeito visual;
- ✓ Realce de informações;
- ✓ Desfoque de bordas;
- ✓ Visão computacional;

4

Filtros espaciais

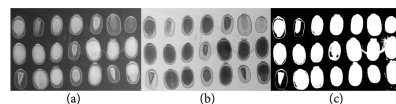
- Também conhecido como processamento ponto a ponto;
- imagem filtrada é obtido por uma equação envolvendo apenas o pixel de mesmas coordenadas espaciais na imagem original.

5

Filtros espaciais

$$g(x, y) = c_{\max} - f(x, y)$$

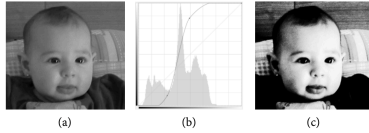
$$g(x, y) = c_{\max}, \text{ se } f(x, y) > I, \text{ e } g(x, y) = 0$$



Fonte: Saúde (2018, p.152)

6

Curva de Cores

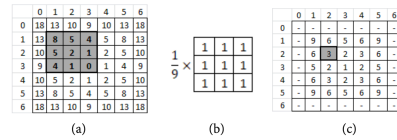


Fonte: Saúde (2019, p.153)

7

Convolução e correlação

Cálculo do pixel de acordo com o valor dos pixels vizinhos;

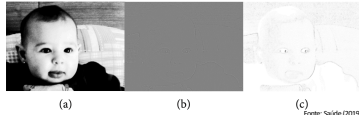


Fonte: Saúde (2019, p.155)

8

Filtros por convolução

- Laplaciano: mede a diferença de um pixel com relação à média de seus vizinhos, produzido por uma equação de segunda derivada nos eixos x e y.

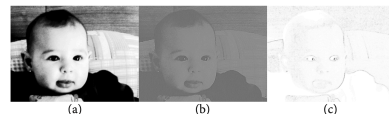


Fonte: Saúde (2019, p.157)

9

Filtros por convolução

- Filtro de aguçamento: realça pixels em regiões de borda.

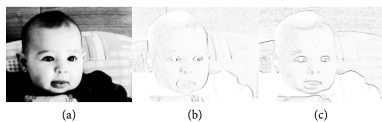


Fonte: Saúde (2019, p.158)

10

Filtros por convolução

- Filtro Sobel: foi construído com base no cálculo da primeira derivada da imagem.

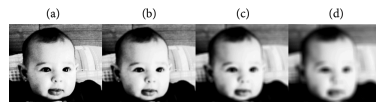


Fonte: Saúde (2019, p.158)

11

Filtros que suavizam (blur)

- Filtro gaussiano: faz-se a média ponderada da vizinhança com base em uma distribuição gaussiana 2D.



Fonte: Saúde (2019, p.159)

12

Pré-processamento de Imagens

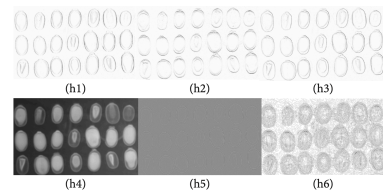
13

- Sua empresa é responsável por levantar características de imagens capturadas por raio X das sementes,
- Você deverá avaliar, para a aplicação em específico, os filtros de processamento de imagens necessários para que a segmentação seja bem-sucedida e você possa entregar o produto requerido para a terceira etapa do trabalho.

14

```
f = np.asarray(Image.open("semente.png"))
h1 = invert(normaliza(abs(filtroSobelLR(f))))
h2 = invert(normaliza(abs(filtroSobelUD(f))))
h3 = invert(normaliza(abs(filtroSobelLR(f)+abs(filtroSobelUD(f)))))
h4 = filtroGaussiano(f,15).astype(np.uint8)
h5 = normaliza(filtroLaplaciano(f))
h6 = invert(normaliza(abs(filtroLaplaciano(h4))))
```

15



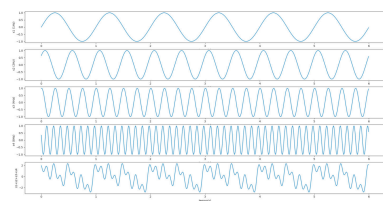
Fonte: Salde (2005, p.180)

16

Filtros no Domínio da Frequência

17

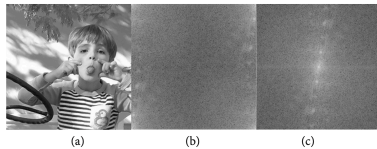
Série de Fourier



Fonte: Salde (2005, p.167)

18

Série de Fourier



Fonte: Saúde (2019, p.189)

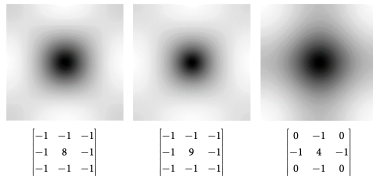
19

Filtros no domínio de Frequência

As frequências mais baixas estão próximas ao centro das imagens. Esses filtros são passa-alta, porque atenuam, as frequências baixas e destacam as altas frequências.

20

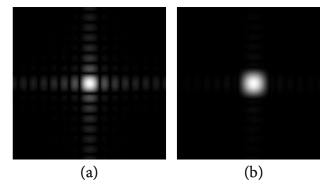
Filtros no domínio de Frequência



Fonte: Saúde (2019, p.172)

21

Filtros no domínio de Frequência

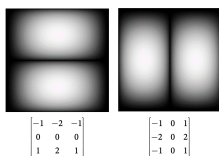


Fonte: Saúde (2019, p.172)

22

Filtros no domínio de Frequência

- Filtro Sobel: filtros passa-alta, mas com realces diferentes na horizontal e na vertical.



Fonte: Saúde (2019, p.173)

23

Filtros no Domínio de Frequência

24

- Demonstrar a aplicação, no domínio da frequência, dos mesmos filtros até então aplicados no domínio espacial.
- Você deve entregar um relatório mostrando a diferença da aplicação dos filtros no domínio da frequência com relação à aplicação no domínio espacial, com o código fonte que permite a criação do relatório.

25

```
def kernelSobelUD(f):
    return np.array([[[-1,-2,-1],
                      [ 0, 0, 0],
                      [ 1, 2, 1]])

def kernelSobelLR(f):
    return np.array([[[-1,0,1],
                      [-2,0,2],
                      [-1,0,1]])
```

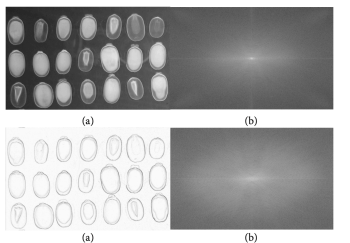
26

```
def filtroTeoConv(f,g):
    (gy,gx) = g.shape
    gg = np.zeros(f.shape) # filtro nas dimensões de f
    gg[0:gy,0:gx] = g
    F = np.fft.fft2(f)
    G = np.fft.fft2(gg)
    return np.fft.ifft2(F*G)
```

27

```
f = np.asarray(Image.open('semente.png'))
sobelLR = abs(filtroTeoConv(f, kernelSobelLR(f)))
sobelUD = abs(filtroTeoConv(f, kernelSobelUD(f)))
hS = invert(normaliza(sobelLR+sobelUD))
```

28



Fonte: Salde (2013, p.176)

29

Para qual finalidade podemos utilizar os filtros do tipo *blur*?

30

Segmentação de Imagens

31

Segmentação

- É o processo que subdivide uma imagem em objetos ou partes que a constituem.
- Realiza a detecção e o delineamento de regiões de interesse.

32

Técnicas de Segmentação

O que considerar?

- Precisão;
- Tempo de processamento;
- Eficácia da técnica para as características da imagem a ser segmentada;

33

Técnicas de Segmentação

- **Baseada em Regiões:** buscam associar, em uma mesma região, pixels que se pareçam, de acordo com critérios de similaridade.
- **Detecção de Bordas:** buscam a dissimilaridade, que são os pixels da fronteira ou separação entre regiões.

34

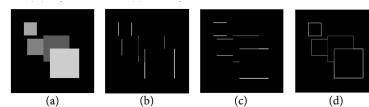
Segmentação baseada em gradiente

- Detecção das dissimilaridades no entorno de cada ponto da imagem.
- O vetor $\nabla f_p = \left\langle \frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right\rangle$ é um vetor de derivadas parciais da função $f(x,y)$ para o ponto $p=(x_1,y_1)$.

35

Segmentação baseada em gradiente

O filtro de Sobel calcula uma aproximação da norma do gradiente em cada ponto da imagem, mas cada rotação do filtro de Sobel aproxima a derivada parcial em uma direção.



Fonte: Salade (2003, p.184)

36

Segmentação por similaridade

- Consiste na agregação, em um mesmo objeto, dos pixels com características similares.
- O objetivo é marcar todos os pixels que compõem o objeto, e não o seu contorno.

37

Segmentação por similaridade

- Aplicação de um Limiar



Fonte: Saúde (2019, p.185)

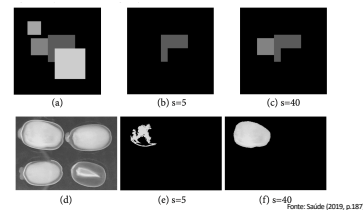
38

Segmentação por similaridade

- Crescimento de Regiões: consiste em marcar um ou mais pontos do objeto de interesse e, a partir desses pontos de referência, agregar ao objeto os pontos similares com base nas vizinhanças dos pixels.

39

Segmentação por similaridade



Fonte: Saúde (2019, p.187)

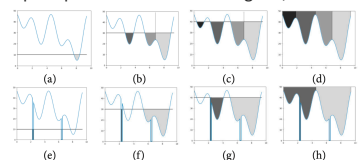
40

Watershed

41

Watershed

- Técnica de segmentação de imagens baseado no princípio de crescimento de regiões;



Fonte: Saúde (2019, p.187)

42

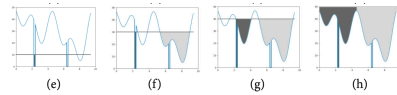
Watershed

- Conceito de inundações refere-se ao crescimento de regiões.
- A diferença é que, no crescimento de regiões, dois pixels vizinhos são colocados na mesma região por um critério de similaridade dos seus níveis de cinza, enquanto no watershed os pixels são colocados na mesma região se pertencem ao mesmo vale ou bacia hidrográfica.

43

Watershed

- Cada mínimo local se torna uma região;
- Watershed por marcadores



Fonte: Salda (2019, p.187)

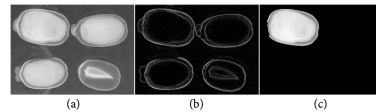
44

Implementação do Watershed

- Para cada marcador, atribuiu-se um rótulo, e desses marcadores começa o crescimento.
- É necessário usar uma estrutura de dados de *heap* para implementar uma fila de prioridades.
- O pixel que crescerá será sempre o pixel de menor nível de cinza.
- O nível de cinza mais baixo é a prioridade.

45

Watershed



Fonte: Salda (2019, p.188)

46

Watershed

47

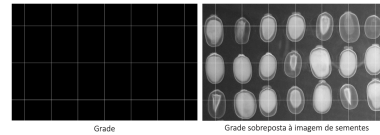
- Agora, você deve aplicar um algoritmo de segmentação para finalizar o produto do seu trabalho.
- Demonstre que é possível segmentar as sementes com o algoritmo de watershed.
- Você deve entregar a imagem segmentada e o código fonte.

48

Primeiro deve-se fazer o reconhecimento de que as sementes de café estão bem alinhadas em uma grade.

```
def grade():
    # grade
    yy = []
    for i in range(120,600,200):
        yy.append(i)
    xx = []
    for i in range(70,1010,145):
        xx.append(i)
    return (yy,xx)
```

49



Fonte: Saide (2019, p.191)

50

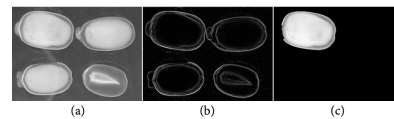
```
import heapq as hq
def watershed(f,marcadores):
    # cria pq e saída
    h = []; w = np.zeros(f.shape)

    # insere marcadores na pq, com prioridade 0
    for (y,x) in np.argwhere(marcadores>0):
        w[y,x] = marcadores[y,x]
        hq.heappush(h,(0,[y,x]))

    # iterações
    while len(h) > 0:
        (_,(y,x)) = hq.heappop(h)
        insereVizinhos(f,w,h,y,x)

    return w
```

51



Fonte: Saide (2019, p.191)

52

Filtros de Imagens

53

Dúvidas?

54

Recapitulando

55

Filtros de Imagens

- Filtros Espaciais;
- Convolução e correlação;
- Filtros por convolução
- Filtros *blur*;
- Filtros no Domínio da Frequência

56

Segmentação de Imagens

- Baseadas em Regiões;
- Detecção de Bordas;
- Watershad

57



58